

Functions

Writing your own functions

So far we've seen many functions, like `c()`, `class()`, `filter()`, `dim()` ...

Why create your own functions?

- Cut down on repetitive code (easier to fix things!)
- Organize code into manageable chunks
- Avoid running code unintentionally
- Use names that make sense to you

Writing your own functions

The general syntax for a function is:

```
my_function <- function(argument) {  
  <function body>  
}
```

OR

```
my_function <- \(argument) {  
  <function body>  
}
```

Writing your own functions

Here we will write a function that divides some number x by 100:

```
div_100 <- function(x) x / 100
```

When you run the line of code above, you make it ready to use (no output yet!).

Let's test it:

```
div_100(x = 600)
```

```
[1] 6
```

Writing your own functions

We can take all kinds of arguments. Here is one with text:

```
greeting <- function(name) paste("Hello my name is", name)
```

Let's test it:

```
greeting(name = "Ava") # Named argument
```

```
[1] "Hello my name is Ava"
```

```
greeting("Ava") # R guesses argument based on position
```

```
[1] "Hello my name is Ava"
```

Writing your own functions: { }

Adding the curly braces - {} - allows you to use functions spanning multiple lines:

```
div_100 <- function(x) {  
  x / 100  
}  
div_100(x = 10)  
  
[1] 0.1
```

Writing your own functions: return

If we want something specific for the function's output, we use `return()`:

```
div_100_plus_4 <- function(x) {  
  output_int <- x / 100  
  output <- output_int + 4  
  return(output)  
}  
div_100_plus_4(x = 10)
```

```
[1] 4.1
```

Writing your own functions: multiple inputs

Functions can take multiple inputs:

```
div_100_plus_y <- function(x, y) x / 100 + y  
div_100_plus_y(x = 10, y = 3)
```

```
[1] 3.1
```

Writing your own functions: multiple outputs

Functions can return a vector (or other object) with multiple outputs.

```
x_and_y_plus_2 <- function(x, y) {  
  output1 <- x + 2  
  output2 <- y + 2  
  
  return(c(output1, output2))  
}  
result <- x_and_y_plus_2(x = 10, y = 3)  
result  
  
[1] 12 5
```

Writing your own functions: defaults

Functions can have “default” arguments. This lets us use the function without using an argument later:

```
div_100_plus_y <- function(x = 10, y = 3) x / 100 + y  
div_100_plus_y()
```

```
[1] 3.1
```

```
div_100_plus_y(x = 11, y = 4)
```

```
[1] 4.11
```

Writing another simple function

Let's write a function, `sqdif`, that:

1. takes two numbers `x` and `y` with default values of 2 and 3.
2. takes the difference
3. squares this difference
4. then returns the final value

Writing another simple function

```
sqdif <- function(x = 2, y = 3) (x - y)^2
```

```
sqdif()
```

```
[1] 1
```

```
sqdif(x = 10, y = 5)
```

```
[1] 25
```

```
sqdif(10, 5)
```

```
[1] 25
```

```
sqdif(11, 4)
```

```
[1] 49
```

Writing your own functions: characters

Again, functions can have any kind of input.

```
loud <- function(word = "hooray!") {  
  output <- rep(toupper(word), 5)  
  return(output)  
}
```

```
loud()
```

```
[1] "HOORAY!" "HOORAY!" "HOORAY!" "HOORAY!" "HOORAY!"
```

```
loud("wow!")
```

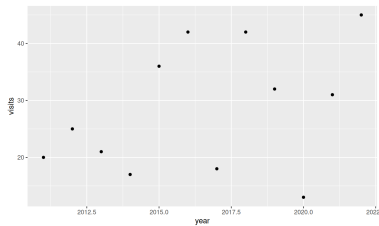
```
[1] "WOW!" "WOW!" "WOW!" "WOW!" "WOW!"
```

Functions for tibbles - Example with ggplot

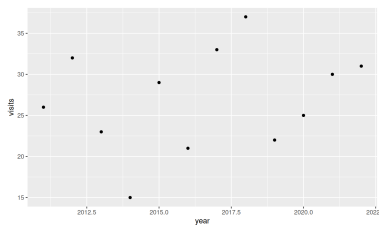
```
er <- read_csv(file = "https://daseh.org/data/CO_ER_heat_visits.csv")
```

```
visits_plot <- function(the_county){  
  er_sub <- er |> filter(county == the_county)  
  ggplot(data = er_sub, aes(x = year, y = visits)) +  
    geom_point()  
}
```

```
visits_plot("Larimer")
```



```
visits_plot("Weld")
```



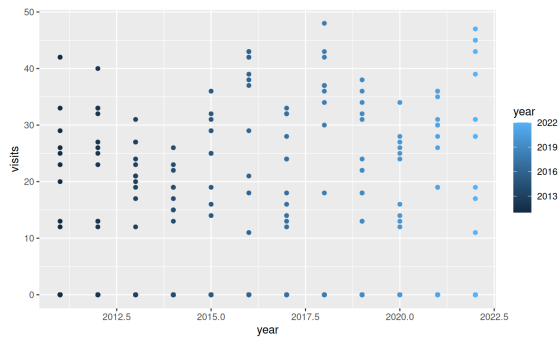
Functions for tibbles - curly braces

Tell tidyverse functions that you mean the **column** not an **object** with curly braces ({}):

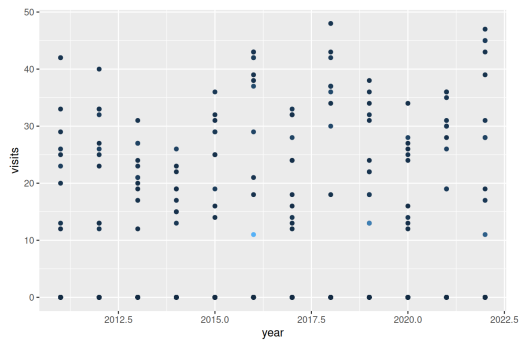
```
visits_plot2 <- function(the_column){  
  ggplot(data = er, aes(x = year, y = visits, color = {{the_column}})) +  
    geom_point()  
}
```

Functions for tibbles - example

`visits_plot2(year)`



`visits_plot2(rate)`



Summary

- Simple functions take the form:
 - `NEW_FUNCTION <- function(x, y){x + y}`
 - Can specify defaults like `function(x = 1, y = 2){x + y}`
 - `return` will provide a value as output
- Specify a column (from a tibble) inside a function using `{{double curly braces}}`

Lab Part 1

▮ [Class Website](#)

▮ [Lab](#)

Functions on multiple columns

Using your custom functions: **sapply()**- a base R function

Now that you've made a function... You can “apply” functions easily with **sapply()**!

These functions take the form:

```
sapply(<a vector, list, data frame>, some_function)
```

Using your custom functions: `sapply()`

Let's apply a function to look at the CO heat-related ER visits dataset.

▯ There are no parentheses on the functions!▯

You can also pipe into your function.

```
sapply(er, class)
```

county	rate	lower95cl	upper95cl	visits	year
"character"	"numeric"	"numeric"	"numeric"	"numeric"	"numeric"

```
# also: er |> sapply(class)
```

Using your custom functions: `sapply()`

Use the `div_100` function we created earlier to convert 0-100 percentiles to proportions.

```
er |>  
  select(ends_with("c1")) |>  
  sapply(div_100) |>  
  head()
```

	lower95c1	upper95c1
[1,]	NA	0.09236776
[2,]	0.02848937	NA
[3,]	0.04359735	0.09313561
[4,]	0.01711087	0.04846996
[5,]	0.01892912	0.05232461
[6,]	0.06124961	0.11572046

Using your custom functions “on the fly” to iterate

Also called “anonymous function”.

```
er |>  
  select(ends_with("c1")) |>  
  sapply(function(x) x / 100) |>  
  head()
```

	lower95c1	upper95c1
[1,]	NA	0.09236776
[2,]	0.02848937	NA
[3,]	0.04359735	0.09313561
[4,]	0.01711087	0.04846996
[5,]	0.01892912	0.05232461
[6,]	0.06124961	0.11572046

Anonymous functions: alternative syntax

```
er |>  
  select(ends_with("c1")) |>  
  sapply(\(x) x / 100) |>  
  head()
```

	lower95c1	upper95c1
[1,]	NA	0.09236776
[2,]	0.02848937	NA
[3,]	0.04359735	0.09313561
[4,]	0.01711087	0.04846996
[5,]	0.01892912	0.05232461
[6,]	0.06124961	0.11572046

across

Using functions in `mutate()` and `summarize()`

Already know how to use functions to modify columns using `mutate()` or calculate summary statistics using `summarize()`.

```
er |>
  summarize(max_visits = max(visits, na.rm = T),
            max_rate = max(rate, na.rm = T))
```

```
# A tibble: 1 × 2
  max_visits max_rate
    <dbl>     <dbl>
1         48      89.3
```

Applying functions with **across** from **dp1yr**

`across()` makes it easy to apply the same transformation to multiple columns. Usually used with `summarize()` or `mutate()`.

```
summarize(across(<columns>, function))
```

or

```
mutate(across(<columns>, function))
```

- List columns first: `.cols =`
- List function next: `.fns =`
- If there are arguments to a function (e.g., `na.rm = TRUE`), use an anonymous function.

Applying functions with **across** from **dplyr**

Combining with `summarize()`

```
er |>
  summarize(across(
    c(visits, rate),
    mean # no parentheses
  ))
```

```
# A tibble: 1 × 2
  visits rate
  <dbl> <dbl>
1      NA   NA
```

Applying functions with **across** from **dplyr**

Add anonymous function to include additional arguments (e.g., `na.rm = T`).

```
er |>
  summarize(across(
    c(visits, rate),
    function(x) mean(x, na.rm = T)
  ))
```

```
# A tibble: 1 × 2
  visits rate
  <dbl> <dbl>
1    7.19  2.43
```

Applying functions with **across** from **dplyr**

Can use with other tidyverse functions like `group_by`!

```
er |>
  group_by(year) |>
  summarize(across(
    c(visits, rate),
    function(x) mean(x, na.rm = T)
  ))
```

```
# A tibble: 12 × 3
   year visits  rate
  <dbl> <dbl> <dbl>
1  2011   5.20  1.49
2  2012   5.89  1.75
3  2013   5.63  1.83
4  2014   4.12  1.41
5  2015   6.4   1.96
6  2016  10.1   5.28
7  2017   7.24  2.13
8  2018  11.7   3.28
9  2019   9.12  4.09
10 2020   6.26  1.73
11 2021   8.06  2.08
12 2022   9.29  3.21
```

Applying functions with **across** from **dplyr**

Using different `tidyselect()` options (e.g., `starts_with()`, `ends_with()`, `contains()`)

```
er |>
  group_by(year) |>
  summarize(across(
    contains("cl"),
    function(x) mean(x, na.rm = TRUE)
  ))
```

```
# A tibble: 12 × 3
  year lower95cl upper95cl
  <dbl>      <dbl>      <dbl>
1  2011      0.836      2.12
2  2012      1.06      2.41
3  2013      1.07      2.62
4  2014      0.810      2.11
5  2015      1.21      2.77
6  2016      3.05      7.99
7  2017      1.28      3.08
8  2018      2.17      4.41
9  2019      2.32      6.21
10 2020      1.02      2.52
11 2021      1.30      2.92
12 2022      1.93      4.71
```

Applying functions with **across** from **dp1yr**

Combining with `mutate()` - the `replace_na` function

Let's look at the yearly CO2 emissions dataset.

```
yearly_co2 <-  
  read_csv(file = "https://daseh.org/data/Yearly_CO2_Emissions_1000_tonnes.csv")
```

```
yearly_co2 |>  
  select(country, starts_with("194")) |>  
  mutate(across(  
    c(`1943`, `1944`, `1945`),  
    function(x) replace_na(x, replace = 0)  
  ))
```

```
# A tibble: 192 × 11
```

	country	`1940`	`1941`	`1942`	`1943`	`1944`	`1945`	`1946`	`1947`	`1948`
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	Afghanistan	NA	NA	NA	0	0	0	NA	NA	NA
2	Albania	693	627	744	462	154	121	484	928	704
3	Algeria	238	312	499	469	499	616	763	744	803
4	Andorra	NA	NA	NA	0	0	0	NA	NA	NA
5	Angola	NA	NA	NA	0	0	0	NA	NA	NA
6	Antigua and B...	NA	NA	NA	0	0	0	NA	NA	NA
7	Argentina	15900	14000	13500	14100	14000	13700	13700	14500	17400
8	Armenia	848	745	513	655	613	649	730	878	935
9	Australia	29100	34600	36500	35000	34200	32700	35500	38000	38500
10	Austria	7350	7980	8560	9620	9400	4570	12800	17600	24500

```
#   182 more rows
```

```
#   1 more variable: `1949` <dbl>
```

GUT CHECK!

Why use `across()`?

- A. Efficiency - faster and less repetitive
- B. Calculate the cross product
- C. Connect across datasets

purrr package

Similar to `across`, `purrr` is a package that allows you to apply a function to multiple columns in a data frame or multiple data objects in a list.

While we won't get into `purrr` too much in this class, it's part of the tidyverse and is great if you're doing lots of iterative work!

One cool function: `modify_if()`

Columns must meet a certain criteria to be modified.

```
er |>  
  modify_if(is.numeric, \(x) round(x)) |>  
  glimpse()
```

Rows: 768

Columns: 6

```
$ county    <chr> "Adams", "Adams", "Adams", "Adams", "Adams", "Adams", "Adams...  
$ rate      <dbl> 7, 5, 7, 3, 3, 9, 7, 7, 7, 5, 7, 8, 0, 0, NA, 0, NA, NA, NA,...  
$ lower95cl <dbl> NA, 3, 4, 2, 2, 6, 4, 5, 5, 3, 5, 6, 0, 0, NA, 0, NA, NA, NA..  
$ upper95cl <dbl> 9, NA, 9, 5, 5, 12, 9, 9, 9, 7, 9, 11, 0, 0, NA, 0, NA, NA, ...  
$ visits    <dbl> 29, 23, 31, 15, 16, 42, 32, 37, 36, 24, 35, 45, 0, 0, NA, 0,...  
$ year      <dbl> 2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018, 2019, 2020, ...
```

Summary

- Apply your functions with `sapply(<a vector or list>, some_function)`
- Use `across()` to apply functions across multiple columns of data
- Need to use `across` within `summarize()` or `mutate()`
- Check out `purrr` if you want to get into the weeds!

Lab Part 2

- ▮ [Class Website](#)
- ▮ [Lab](#)
- ▮ [Day 9 Cheatsheet](#)
- ▮ [Posit's purrr Cheatsheet](#)

Research Survey

<https://forms.gle/5ac2WPiR9kJu5D3c6>



Image by [Gerd Altmann](#) from [Pixabay](#)